



NATIONAL UNIVERSITY
of Computer & Emerging Sciences

Name: Muhammad Musawar Ali Shah

Roll No. 24P-0619

Dept: BS-CS

Section: BCS-3C

Subject: Data Structures

Submitted To:

Sir Samin Ahmed

Assignment 1

Question 1:

Code

```
#include<iostream>
using namespace std;
class Node {
public:
    char data;
    Node *prev;
    Node *next;
    Node(char val) : data(val), prev(NULL) , next(NULL) {}
};
class Line{
public:
    Node *Head, *Tail;
    Line *next, *prev;
    Line() : Head(NULL), Tail(NULL), next(NULL), prev(NULL) {}

    void insertCharAt(Node* i, char val) {
        Node *newChar = new Node(val);

        if(i == NULL) {
            if(Head != NULL) {
                newChar->next = Head;
                Head->prev = newChar;
            } else Tail = newChar;

            Head = newChar;
        } else {
            newChar->next = i->next;
            newChar->prev = i;

            if(i->next != NULL) i->next->prev = newChar;
            else Tail = newChar;
            i->next = newChar;
        }
    }

    void deleteCharAt(Node* i) {
        if(i == NULL) return;
        if(i->prev != NULL) i->prev->next = i->next;
        else Head = i->next;

        if(i->next != NULL) i->next->prev = i->prev;
        else Tail = i->prev;
    }
}
```

```

        delete i;
    }

    int getCharCount() {
        int count = 0;
        Node *temp = Head;
        while(temp != NULL) {
            count++;
            temp = temp->next;
        }
        return count;
    }
};

class TextEditor{
private:
    Line *docHead, *docTail, *currLine;
    Node *cursor;
public:
    TextEditor() {
        docHead = docTail = currLine = new Line();
        cursor = NULL;
    }

    void moveCursorLeft() {
        if(cursor != NULL && cursor->prev != NULL) cursor = cursor->prev;
        else if(cursor == NULL && currLine->Tail != NULL) {
            if(currLine->prev != NULL) {
                currLine = currLine->prev;
                cursor = currLine->Tail;
            }
        } else if(cursor != NULL) cursor = NULL;
    }

    void moveCursorRight() {
        if(cursor == NULL && currLine->Head != NULL) cursor = currLine->Head;
        else if(cursor != NULL && cursor->next != NULL) cursor = cursor->next;
        else {
            if(currLine->next != NULL) {
                currLine = currLine->next;
                cursor = NULL;
            }
        }
    }

    void moveCursorUp() {
        if(currLine->prev == NULL) return;
        int curr = 0;

```

```

Node *temp = currLine->Head;
while(temp != NULL && temp != cursor) {
    curr++;
    temp = temp->next;
}

currLine = currLine->prev;

cursor = NULL;
temp = currLine->Head;
for(int i = 0 ; i < curr && temp; i++) {
    cursor = temp;
    temp = temp->next;
}
}

void moveCursorDown() {
    if(currLine->next == NULL) return;

    int curr = 0;
    Node *temp = currLine->Head;
    while (temp != NULL && temp != cursor) {
        curr++;
        temp = temp->next;
    }

    currLine = currLine->next;

    cursor = NULL;
    temp = currLine->Head;
    for(int i = 0; i < curr && temp; i++) {
        cursor = temp;
        temp = temp->next;
    }
}

void insertChar(char val) {
    if(val == '\n') {
        Line *newLine = new Line();

        if(cursor != NULL && cursor->next != NULL) {
            newLine->Head = cursor->next;
            newLine->Tail = currLine->Tail;
            newLine->Head->prev = NULL;

            currLine->Tail = cursor;
            cursor->next = NULL;
        }
    }
}

```

```

newLine->next = currLine->next;
newLine->prev = currLine;

if(currLine->next != NULL) currLine->next->prev = newLine;
else docTail = newLine;
currLine->next = newLine;

currLine = newLine;
cursor = NULL;

} else {
    currLine->insertCharAt(cursor, val);
    if (cursor != NULL) cursor = cursor->next;
    else cursor = currLine->Head;
}
}

void backspace() {
    if(cursor != NULL) {
        Node *deletion = cursor;
        cursor = cursor->prev;
        currLine->deleteCharAt(deletion);
    } else if(currLine->prev != NULL) {
        Line *prev = currLine->prev;
        cursor = prev->Tail;

        if(prev->Tail != NULL && currLine->Head) {
            prev->Tail->next = currLine->Head;
            currLine->Head->prev = prev->Tail;
            prev->Tail = currLine->Tail;
        } else if(currLine->Head) {
            prev->Head = currLine->Head;
            prev->Tail = currLine->Tail;
        }

        if(currLine->next) currLine->next->prev = prev;
        else docTail = prev;

        prev->next = currLine->next;

        delete currLine;
        currLine = prev;
    }
}

void deleteChar() {
    if(cursor != NULL && cursor->next != NULL) currLine->deleteCharAt(cursor->next);
    else if(cursor == NULL && currLine->Head != NULL) {

```

```

        Node* deletion = currLine->Head;
        currLine->deleteCharAt(deletion);
    } else if(currLine->next != NULL) {
        Line* nextLine = currLine->next;

        if(currLine->Tail != NULL && nextLine->Head != NULL) {
            currLine->Tail->next = nextLine->Head;
            nextLine->Head->prev = currLine->Tail;
            currLine->Tail = nextLine->Tail;
        } else if(nextLine->Head != NULL) {
            currLine->Head = nextLine->Head;
            currLine->Tail = nextLine->Tail;
        }

        if(nextLine->next != NULL) nextLine->next->prev = currLine;
        else docTail = currLine;

        currLine->next = nextLine->next;

        delete nextLine;
    }
}

```

```

int searchChar(char val) {
    int count = 0;

    Line* line = docHead;

    while(line != NULL) {
        Node* ch = line->Head;
        while(ch != NULL) {
            if(ch->data == val) count++;

            ch = ch->next;
        }
        line = line->next;
    }
    return count;
}

```

```

void display() {
    Line* line = docHead;
    int num = 1;
    while(line != NULL) {
        cout<<"Line "<<num<<": ";
        Node* ch = line->Head;

        while(ch != NULL) {

```

```

        if(ch == cursor) cout<<"|";
        cout<<ch->data;
        ch = ch->next;
    }

    if(cursor == NULL && line == currLine) cout<<"|";
    cout<<endl;

    line = line->next;
    num++;
}
cout<<"---"<<endl;
}

~TextEditor() {
    Line *line = docHead;
    while(line != NULL) {
        Node *ch = line->Head;
        while(ch != NULL) {
            Node *temp = ch->next;
            delete ch;
            ch = temp;
        }

        Line *nextLine = line->next;
        delete line;
        line = nextLine;
    }
}

};

int main() {
    TextEditor E1;

    E1.insertChar('H');
    // E1.display();
    E1.insertChar('E');
    // E1.display();
    E1.insertChar('L');
    E1.insertChar('L');
    E1.insertChar('O');
    E1.display();

    cout << "After inserting HELLO:" << endl;
    E1.display();

    E1.moveCursorLeft();

```

```

E1.moveCursorLeft();
E1.insertChar('A');
E1.display();

E1.moveCursorLeft();
E1.moveCursorLeft();
E1.moveCursorLeft();
E1.moveCursorLeft();
E1.display();

E1.insertChar('#');
E1.display();

E1.moveCursorRight();
E1.moveCursorRight();
E1.moveCursorRight();
E1.display();

E1.insertChar('\n');
E1.insertChar('W');
E1.insertChar('O');
E1.insertChar('R');
E1.insertChar('L');
E1.insertChar('D');
E1.display();

E1.backspace();
E1.display();

E1.moveCursorUp();
E1.display();

E1.moveCursorDown();
E1.display();

E1.searchChar('O');
return 0;
}

```

Question 2:

Code

```

#include<iostream>
using namespace std;
class Block{
public:

```

```

int BlockID, WorkDone, PrevKey, CurrKey;
string data;
Block *next1, *next2;
Block(int id,string d,int work,int prev) : BlockID(id), data(d), WorkDone(work), PrevKey(prev),
next1(NULL), next2(NULL) {
    int sum = 0;
    for(char c: data) sum+= (int)c;
    CurrKey = (BlockID * 27 + WorkDone * 19 +sum + PrevKey) % 100000;
}
};
class Blockchain{
private:
    Block *Head;
public:
    Blockchain() : Head(NULL) {}
    void createGenesisBlock() {
        Head = new Block(1,"",0,0);
    }

    void addBlock(string data, int work) {
        if(Head == NULL) {
            createGenesisBlock();
            return;
        }
        Block *temp = Head;
        while(temp->next1 != NULL) temp = temp->next1;
        Block *newBlock = new Block(temp->BlockID + 1,data,work, temp->CurrKey);
        temp->next1 = newBlock;
    }

    void forkChain(int blockID, string data1, string data2, int work1,int work2) {
        Block *temp = Head;
        while(temp != NULL && temp->BlockID != blockID) temp = temp->next1;

        if(temp == NULL) {
            cout<<"Block not found"<<endl;
            return;
        }
        temp->next1 = new Block(blockID + 1,data1, work1, temp->CurrKey);
        temp->next2 = new Block(blockID + 1,data2, work2, temp->CurrKey);
    }

    void tamperBlock(int blockID, int newKey) {
        Block *temp = Head;
        while(temp != NULL) {
            if(temp->BlockID == blockID) {
                temp->CurrKey = newKey;
                return;
            }
        }
    }
}

```

```

    }
    temp = temp->next1;
}

if(temp == NULL || temp->BlockID != blockID) {
    cout<<"Block not found"<<endl;
    return;
}
temp->CurrKey = newKey;
}

void repairChain(int blockID) {
    Block *temp = Head;
    while(temp != NULL && temp->BlockID != blockID) temp = temp->next1;

    if(temp == NULL) return;
}

void validateChain() {

}

void consensusRule() {

}

void displayAllChains() {
    Block *temp = Head;
    while(temp != NULL) {
        cout<<"Block "<<temp->BlockID<<" , data "<<temp->data<<" , key: "<<temp->CurrKey;

        if(temp->next2 != NULL) cout<<"Fork to block: "<<temp->next2->BlockID;

        cout<<endl;
        temp = temp->next1;
    }

    temp = Head;
    while(temp != NULL) {
        if(temp->next2 != NULL) {
            cout<<"Fork from block "<<temp->BlockID<<":"<<endl;
            Block *fork = temp->next2;
            while(fork != NULL) {
                cout<<" Block "<<fork->BlockID<<" Data: "<<fork->data<<" , Key: "<<fork->CurrKey<<endl;

                fork = fork->next1;
            }
        }
    }
}

```

```
        temp = temp->next1;
    }
}
};
```

```
int main() {
    Blockchain B1;

    B1.createGenesisBlock();
    B1.addBlock("Task 1", 10);
    B1.addBlock("Task 2", 20);

    B1.forkChain(2, "Fork A", "Fork B", 15, 25);

    B1.displayAllChains();
    B1.validateChain();

    B1.tamperBlock(2, 99999);
    B1.validateChain();

    B1.repairChain(2);
    B1.validateChain();

    B1.consensusRule();

    return 0;
}
```